EconForge as a Reproducible Python Toolkit for Steady State Computation in RBC and New Keynesian Models

Finley Dou

Shanghai High School International Division

Abstract

Numerical economics requires more than finding a vector that an optimizer calls successful. Researchers must identify the equations being solved, preserve their economic interpretation, and retain enough evidence to check the result. This paper examines EconForge 0.1.0a1 as a transparent workflow for deterministic steady-state computation. The study reconstructs the released real business cycle and New Keynesian benchmarks, derives their analytical steady states, and traces model parsing, scalar residual evaluation, automatic differentiation, trust-region least squares, and fresh residual verification. Experiments use the published wheel, independently checked against its distribution checksum, in a newly isolated environment. All five released model and starting-point configurations are reproduced and compared with separately evaluated analytical references. Additional bounded experiments examine local residual geometry, analytical Jacobians, and conservative failure handling. Twelve original figures distinguish explanatory schematics from numerical observations. The results support the reproducibility of these particular computations and illustrate why solver termination, residual acceptance, and economic validity must remain separate. The evidence does not establish global convergence, arbitrary-model correctness, stochastic equilibrium validity, or cross-platform certification. EconForge's contribution is an inspectable integration of established numerical methods and explicit verification records. The accompanying scripts, model inputs, reference audit, and figure provenance make the paper suitable for critical human review and provide a limited, reproducible foundation for teaching computational macroeconomics.

Keywords: computational economics; steady state; real business cycles; New Keynesian model; automatic differentiation; nonlinear least squares; reproducibility

1 Introduction

Economic models describe mutually consistent choices and constraints. A household's consumption decision depends on investment returns, firms' production depends on capital, and aggregate investment changes the capital stock available next period. Numerical implementation turns these relationships into equations that a computer can evaluate. That translation is consequential: a sign error, a mistaken time index, or an unreported normalization can produce a plausible-looking answer to the wrong problem. Numerical methods are therefore part of the scientific argument, rather than a final mechanical step (Judd, 1998).

Steady states offer an accessible setting in which to study this argument. A deterministic steady state is a configuration that reproduces itself when the model is held at its stationary reference and innovations are set to their specified steady values. Small examples often permit analytical solutions. These solutions provide an external check on the numerical search, while the software still has to handle variable ordering, nonlinear expressions, derivatives, domains, and acceptance criteria. A benchmark is valuable precisely because its expected answer can be justified independently of the algorithm being tested.

EconForge is an experimental Python toolkit whose first public Alpha supports a narrow workflow: parsing model specifications, evaluating deterministic residuals, computing steady-state Jacobians, and attempting bounded local roots. This paper studies the exact published version 0.1.0a1, rather than an evolving development checkout (Dou, 2026). The software contains additional modules, but their presence in a package is

not evidence that their capabilities belong to the supported public scope. The analysis concerns two real business cycle benchmarks, RBC-A and RBC-B, and a simplified New Keynesian benchmark, NK-A.

The research question is: how can numerical macroeconomic models be represented, solved, and checked within a transparent and reproducible Python workflow? The paper's thesis is that the evidential chain connecting a model input to a qualified result is itself a useful software-methods contribution. EconForge combines established methods with explicit coordinate information, immutable scientific tolerances, structured diagnostics, and a fresh check of the original equations. We assess that integration, without claiming invention of macroeconomic theory, automatic differentiation, or SciPy's algorithms.

The evaluation has three components. First, the actual released equations are reconstructed and their roots derived. Second, the five documented configurations are rerun with the published distribution and pinned scientific dependencies. Third, analytical derivatives and deliberately limited negative tests examine whether the evidence supports the stated result. The associated package includes both successful outcomes and unsuccessful requests. Its intended audience is advanced students and beginning researchers who need to understand what a numerical answer means and how to question it.

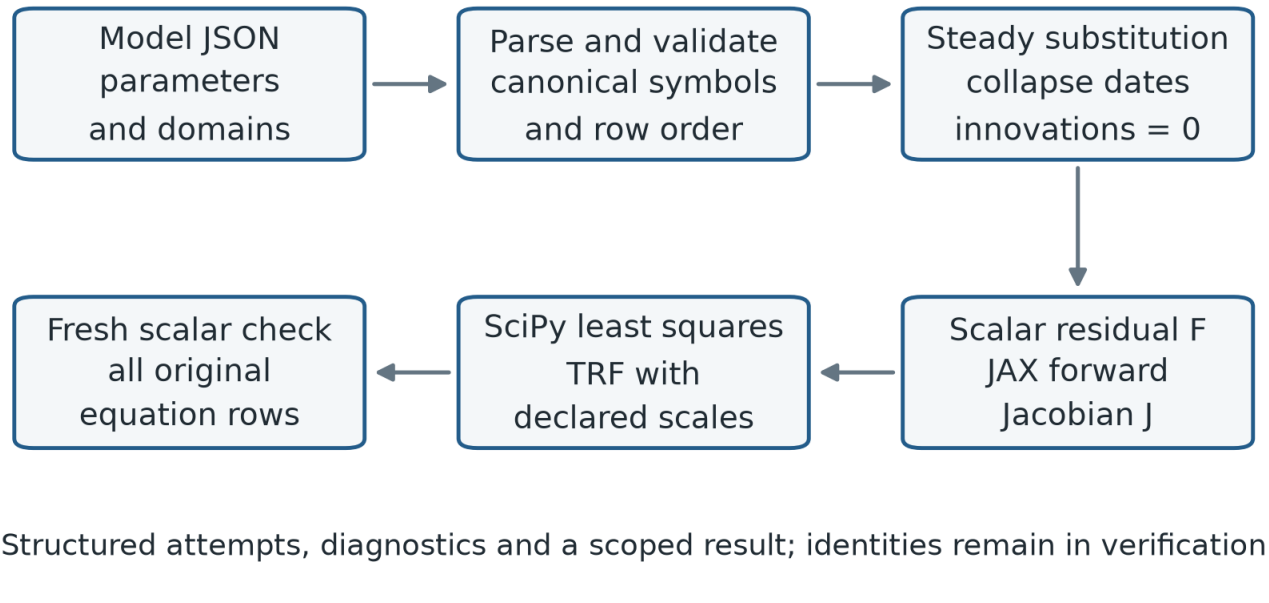


Figure 1. Released steady-state architecture. Released steady-state workflow. Model parsing establishes symbol and equation identities. Dated variables collapse to the supplied steady candidate and innovations are set to zero. The backend receives normalized canonical residuals and an automatic-differentiation Jacobian; a fresh scalar evaluation then checks all original rows, including reporting identities.

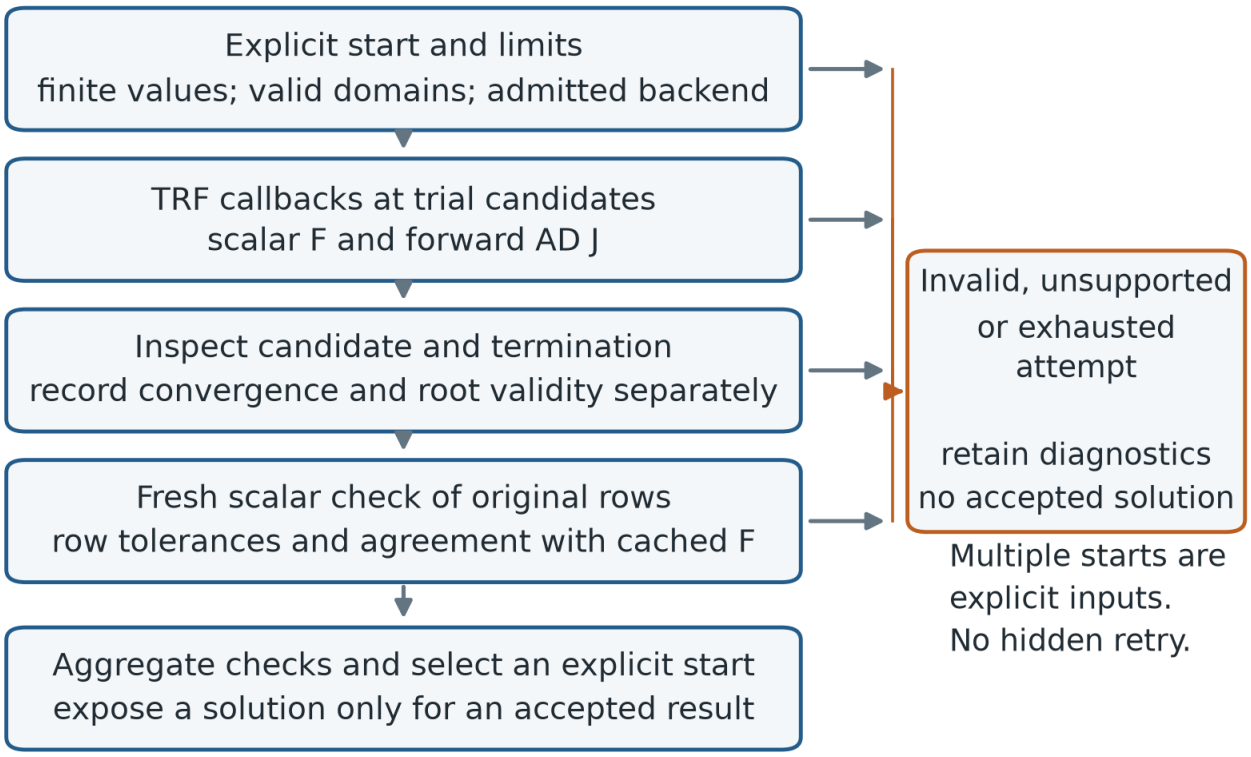


Figure 2. Attempt and acceptance workflow. Attempt and acceptance logic. Input, backend and diagnostic failures are retained rather than repaired silently. A valid backend return is inspected and freshly checked even when termination reports an evaluation limit. Acceptance requires the aggregate checks; a small residual by itself does not replace the convergence obligation.

2 Background and related work

Real business cycle research connects aggregate fluctuations to intertemporal allocation and production. Kydland and Prescott's time-to-build model is a foundational contribution, while King, Plosser, and Rebelo develop the basic neoclassical framework and its growth implications (Kydland and Prescott, 1982; King et al., 1988). Plosser provides a broad exposition of the research program (Plosser, 1989). These works motivate the economic vocabulary used here. The released EconForge examples are much smaller fixed-labor teaching systems; reproducing them does not reproduce the complete quantitative exercises in those papers.

The New Keynesian literature combines intertemporal demand, inflation adjustment, and monetary policy. Clarida, Galí, and Gertler organize these relationships in a framework for analyzing policy, while Taylor studies an influential policy-rule approach (Clarida et al., 1999; Taylor, 1993). NK-A preserves recognizable linear IS, Phillips-curve, and policy relationships. It is not estimated, does not include every mechanism in those studies, and is evaluated here only after deterministic steady substitution. The distinction prevents familiar model labels from implying capabilities that were never exercised.

Computational economics has an extensive numerical-methods foundation (Judd, 1998). SciPy supplies a general numerical computing ecosystem (Virtanen et al., 2020). Its least-squares interface offers a trust-region reflective method for bound-constrained problems; the version used in this study is 1.15.3 (SciPy Developers, 2025). Branch, Coleman, and Li provide relevant background on bound-constrained trust-region methods (Branch et al., 1999). These references explain the method family. The actual EconForge adapter and the versioned library documentation establish the exact options used in this paper.

Automatic differentiation propagates derivatives through the operations that evaluate a function. Unlike a finite-difference approximation, it does not choose a perturbation step and subtract nearby function values. It nevertheless remains subject to floating-point arithmetic and the differentiability of the represented operations (Baydin et al., 2018). JAX provides the derivative transformations used by this release (The JAX Authors, n.d.-a). An automatically computed derivative is a derivative of the implemented expression; it cannot establish that the expression is the intended economic model. Consequently, derivative checks and equation checks remain separate obligations.

Reproducibility research emphasizes recording inputs, versions, computational procedures, and intermediate evidence (Sandve et al., 2013; Wilson et al., 2014). Peng distinguishes computational reproducibility from broader scientific replication, while software citation principles explain why identifying a particular software object matters (Peng, 2011; Smith et al., 2016). The present study applies these ideas at the scale of a small numerical toolkit. A checksum identifies bytes, an environment record identifies execution conditions, and an analytical oracle supports a mathematical comparison. None substitutes for the others.

Dynare offers a much broader documented environment for handling economic models, including tasks beyond steady-state computation (Dynare Team, n.d.). This is a scope comparison based on documentation, not an executed contest between implementations. No external-system runtime, speed comparison, or comparative robustness claim is reported. EconForge's narrower examples are useful here because the path from equations to checked output remains small enough to inspect closely.

3 Mathematical models

3.1 Equilibrium as a residual problem

Let w denote the ordered vector of stored unknowns and θ the parameters. We use w for the general vector to avoid confusing it with the NK output gap x. The canonical steady-state problem is

$$F(w;\theta)=0, w\in\mathcal{D}. \quad (1)$$

Each component of F is the difference between the two sides of an equilibrium restriction. The domain $\mathcal{D}$ contains the permitted values: RBC capital and consumption must remain strictly positive, whereas the declared NK deviations are unbounded. A candidate $\hat{w}$ is a proposed vector. Evaluating its residual is distinct from searching for it. The infinity norm records the largest absolute canonical residual, but row scales are required when comparing equations with different magnitudes.

The released workflow uses positive equation scales S_i and declared magnitudes L_i. It reports row-normalized residuals and checks every original row against the frozen bound

$$\widetilde{F}_i = F_i / S_i, |F_i| \le S_i 10^{-10} + L_i 10^{-8}. \quad (2)$$

Reporting identities also face this check even though they are excluded from the square canonical system. The implementation checks raw and normalized inequalities. These are acceptance contracts for declared scales, not a universal bound on solution error. If a Jacobian is poorly conditioned, a small residual can coexist with a substantial error in the variables.

The steady transformation replaces dated unknowns by the same stored value and innovations by zero. It does not calculate nonlinear conditional expectations. Thus a conditional Euler expression becomes a deterministic stationary restriction, not a computed stochastic expectation. Even an exact root of (1) does not establish optimality, a transversality condition, global stability, empirical fit, or uniqueness across an arbitrary domain. Those are additional economic or mathematical questions.

Table 1. Main notation and implementation conventions.

Symbol	Meaning	Stored convention or role
w, θ	Unknown vector and parameter vector	RBC order (K, z, c); NK order (v, x, π)
K, c, Y, I	Capital, consumption, output, investment	Normalized goods; Y, I are derived observables
$F, \mathscr{D}, U(c)$	Residual vector, valid domain, utility	$F = 0$ defines the root problem; U differs from innovation u
q, R	Marginal product and gross capital return	Appendix A defines $q = \alpha e^{z} K^{\alpha - 1}$ and $R = q + 1 - \delta$
z	Log productivity	Productivity level is $\exp(z)$
$\alpha, \beta, \delta, \gamma$	Capital share, discount factor, depreciation, CRRA curvature	RBC-A hard-codes full depreciation and log marginal utility
ρ, u	Productivity persistence and innovation	Innovation set to zero for steady evaluation
v, x, π, i	Policy disturbance, output gap, inflation, nominal rate	NK deviations; i is an alias, not an unknown
$\sigma, \kappa, \phi_{\pi}, \phi_{x}, \rho_{v}$	IS coefficient, Phillips slope, policy responses, persistence	Declared NK calibration
S_i, L_i, d_j	Equation scale, magnitude, variable scale	Positive numerical metadata
J, D_S, D_d	Raw Jacobian and diagonal scale matrices	$J_{ij} = \partial F_i / \partial w_j$; $D_S = diag(S_i)$; $D_d = diag(d_j)$
$w^{*}, \hat{w}$	Analytical reference and numerical candidate	Asterisk denotes the derived steady state

3.2 RBC-A with fixed labor and full depreciation

RBC-A normalizes labor to a fixed value. Output is produced from capital and productivity, with no endogenous labor-supply equation. Its economic relationships can be written as

$$Y_t = e^{z_t} K_t^{\alpha}, I_t = Y_t - c_t, K_{t+1} = I_t, U(c_t) = \log c_t. \quad (3)$$

All capital depreciates between periods. Consequently, investment must replace the entire capital stock in a steady state. Figure 3 presents these accounting relationships as a schematic; it is not an observation of an economy. The source's exact residual expressions are the resource restriction, a productivity process, and the marginal-utility Euler integrand:

$$K_{t+1} + c_t - e^{z_t} K_t^{\alpha} = 0, z_{t+1} - \rho z_t - u_{t+1} = 0, \quad (4)$$

$$c_t^{-1} - \beta \alpha e^{z_{t+1}} K_{t+1}^{\alpha - 1} c_{t+1}^{-1} = 0. \quad (5)$$

Equation (5) displays the dated marginal-utility integrand; collapsing its dates produces the stationary Euler restriction in (6). A stochastic interpretation would require a conditional expectation around the appropriate future expression. The present API does not perform that expectation. Moreover, RBC-A's JSON contains parameters named delta and gamma, but its literal resource and Euler expressions already specialize to full depreciation and logarithmic marginal utility. Changing unused parameter metadata would not create RBC-B.

The actual steady residuals, in canonical row order, are

$$F_A(K,z,c)=\begin{pmatrix} K+c-e^{z}K^{\alpha} \\ (1-\rho)z \\ c^{-1}\left(1-\beta\alpha e^{z}K^{\alpha-1}\right) \end{pmatrix}. \quad (6)$$

Because $\rho=0.9\neq 1$, the second row requires $z^{*}=0$. Positive consumption lets us divide the Euler row by c^{-1}. Solving the remaining capital restriction and substituting into the resource equation gives

$$K_A^{*}=(\alpha\beta)^{1/(1-\alpha)},\ c_A^{*}=(K_A^{*})^{\alpha}-K_A^{*},\ I_A^{*}=K_A^{*}. \quad (7)$$

With $\alpha=1/3$ and $\beta=0.96$, the reference is $K_A^{*}\approx 0.181019335984$ and $c_A^{*}\approx 0.384666088965$. These values arise from the equations, before calling the solver. The primary start uses 80% of reference capital and 120% of reference consumption; the robustness start reverses those proportions. Both keep log productivity at zero. Their names describe released configurations, not a statistically representative robustness study.

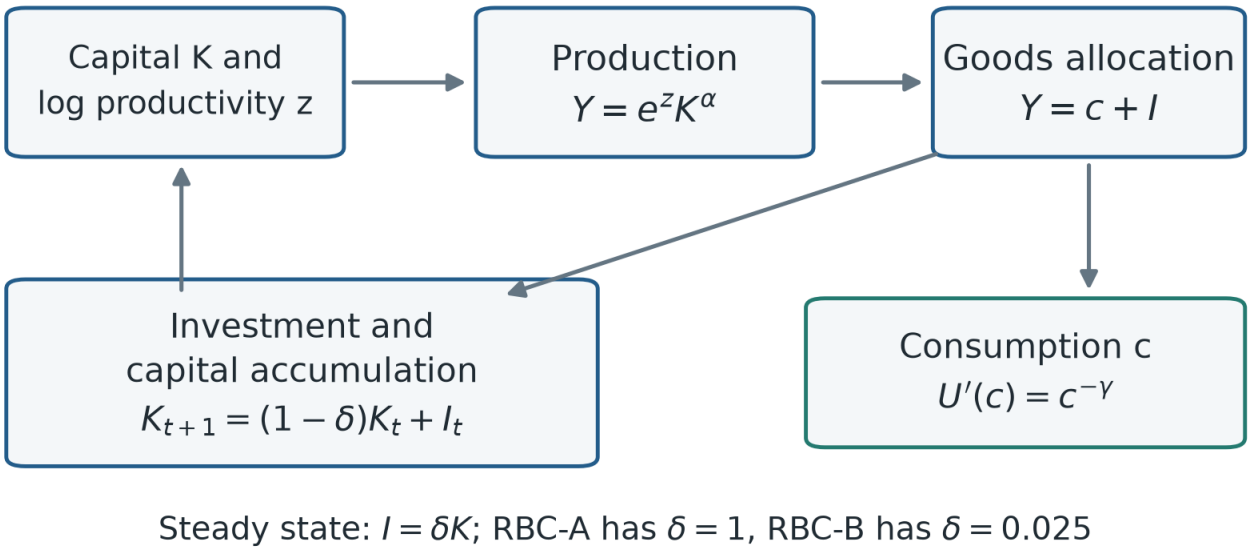


Figure 3. RBC accounting and intertemporal relationships. Economic relationships in the released RBC teaching benchmarks. Production is allocated to consumption and investment. Capital accumulation implies replacement investment I = δK in a steady state, while marginal utility enters the Euler residual. The arrows describe economic accounting, not observed dynamic responses.

3.3 RBC-B with CRRA utility and partial depreciation

RBC-B retains the production relationship but changes both preferences and capital accumulation. With $\gamma=2$, CRRA utility has marginal utility $c^{-\gamma}$. The relevant relationships are

$$U(c)=\frac{c^{1-\gamma}-1}{1-\gamma},\ K_{t+1}=(1-\delta)K_t+I_t. \quad (8)$$

The additive constant in utility has no effect on the marginal condition. The code evaluates the following Euler integrand, together with the resource and productivity restrictions:

$$c_t^{-\gamma} - \beta\, c_{t+1}^{-\gamma}\left(\alpha\, e^{z_{t+1}} K_{t+1}^{\alpha-1} + 1 - \delta\right) = 0. \quad (9)$$

After steady substitution, the model becomes

$$F_B(K, z, c) = \begin{pmatrix} \delta K + c - e^{z} K^{\alpha} \\ (1-\rho) z \\ c^{-\gamma}\left[1 - \beta\left(\alpha\, e^{z} K^{\alpha-1} + 1 - \delta\right)\right] \end{pmatrix}. \quad (10)$$

Capital now survives partly into the next period, so steady investment equals depreciation rather than the whole capital stock. Dividing the Euler equation by positive marginal utility gives

$$K_B^* = \left(\frac{\alpha}{\beta^{-1} - 1 + \delta}\right)^{1/(1-\alpha)}, c_B^* = \left(K_B^*\right)^{\alpha} - \delta K_B^*, I_B^* = \delta K_B^*. \quad (11)$$

At $\alpha = 0.36$, $\beta = 0.99$, $\delta = 0.025$, and $\rho = 0.9$, the reference is $K_B^* \approx 37.9892535382$ and $c_B^* \approx 2.75432747314$. These values use the model's own normalized units. Their magnitude should not be compared with RBC-A as if it were a change in measured wealth under an otherwise identical calibration: several assumptions and parameters change together.

The curvature parameter cancels from the deterministic interior steady-capital equation, but it remains in the residual function away from the root. It therefore affects residual magnitudes and derivatives even when the reference capital is unchanged. This distinction illustrates why equivalent economic roots can be numerically represented differently. Figures 4 and 5 examine that representation on explicitly bounded slices. They establish local shape for the plotted calibration, not a basin of convergence or a result for all admissible parameters.

3.4 NK-A and the policy alias

NK-A uses quarterly, zero-baseline deviations. Its unknown vector is (v, x, π), where v is a monetary-policy disturbance, x is the output gap, and π is inflation. The nominal rate is the alias $i = \phi_\pi \pi + \phi_x x + v$. It is not a fourth independently chosen unknown. The released dated relationships are

$$v_{t+1} - \rho_v v_t - u_{t+1} = 0, x_t - x_{t+1} + (i_t - \pi_{t+1})/\sigma = 0, \quad (12)$$

$$\pi_t - \beta\, \pi_{t+1} - \kappa\, x_t = 0, i_t - \phi_\pi \pi_t - \phi_x x_t - v_t = 0. \quad (13)$$

The final row is a reporting identity once the alias is expanded. The three canonical steady residuals are therefore

$$F_N(v, x, \pi) = \begin{pmatrix} (1-\rho_v) v \\ [v + \phi_x x + (\phi_\pi - 1)\pi]/\sigma \\ (1-\beta)\pi - \kappa\, x \end{pmatrix}. \quad (14)$$

At the calibration in Table 2, $v^* = 0$. The Phillips relation gives $\pi = 10\, x$, whereas the steady IS relation after policy substitution gives $\pi = -x$. Their sole intersection is $x^* = \pi^* = 0$, and hence $i^* = 0$. Figure 6 plots those two eliminated relationships at $v = 0$. The interpretation is a stationary cross-section, not a dynamic policy diagram or an empirical Phillips curve.

The canonical Jacobian is constant:

$$J_N = \begin{pmatrix} 0.5 & 0 & 0 \\ 1 & 0.5 & 0.5 \\ 0 & -0.1 & 0.01 \end{pmatrix}. \quad (15)$$

Its determinant is 0.0275, so this particular linear steady system has a unique root. That is a direct algebraic result for (14), not evidence of dynamic determinacy or a global guarantee for arbitrary nonlinear models. Fresh scalar verification additionally evaluates the fourth, Taylor-identity row; omitting it from the canonical Jacobian does not omit it from the acceptance record.

4 Software design and numerical methodology

4.1 From model text to evaluated equations

The public interface begins with parse_model, which constructs a validated model from bounded JSON. The model records symbols, parameters, expressions, equation roles, domains, scales, and a canonical ordering. The expression grammar is deliberately restricted. Unknown symbols, unsupported constructs, conflicting metadata, and invalid declared domains are rejected rather than delegated to unrestricted execution. This is validation of the representation; it does not prove that a user's economic assumptions are appropriate.

Figure 1 traces the released workflow. A candidate enters scalar residual evaluation and, when needed, automatic differentiation. Model and evaluation identifiers tie these records to particular parameters and candidate values. The solver adapter checks identities and ordering so that an apparently compatible array cannot silently stand in for a derivative from a different point. Such checks address a practical risk in numerical software: arrays can have the correct shape while representing the wrong economic objects.

Stored coordinates deserve particular attention. RBC capital and consumption are levels even though their metadata describe log-deviation coordinates for other uses. The steady root solver works with the stored levels and supplies their declared scales to SciPy; it does not secretly optimize log capital or log consumption. The raw steady Jacobian therefore differentiates with respect to levels. Confusing those derivatives with elasticities would change their meaning. Figure 7 identifies both row and column conventions explicitly.

4.2 Automatic derivatives and the least-squares step

The released AD backend uses JAX in CPU, binary64, no-JIT mode. Explicit 64-bit configuration matters because JAX precision depends on configuration (The JAX Authors, n.d.-b). The production Jacobian is obtained through automatic differentiation of the steady expression, not a finite-difference fallback. The public derivative result includes both the raw Jacobian and its row-normalized form. Its admission checks include finite derivatives and agreement between scalar and JAX primal values; the derivative_valid flag alone is not an independent analytical derivative test. With positive variable scales d_j, the diagnostic matrix is $D_S^{-1} J D_d$. Its singular values describe local numerical sensitivity in the declared scaled coordinates; they do not establish economic validity or a global theorem.

EconForge asks SciPy to solve a bound-constrained nonlinear least-squares problem:

$$\min_{w \in \mathcal{D}} \frac{1}{2} \left\| D_S^{-1} F(w;\theta) \right\|_2^2. \quad (16)$$

The adapter specifies method TRF, linear loss, the exact trust-region subproblem solver, callable AD derivatives, and declared variable scales. The word exact identifies SciPy's subproblem option, not an exact-arithmetic or exact-root claim (SciPy Developers, 2025). All three optimizer termination tolerances are 10^{-12}.

The objective uses a squared Euclidean norm; the reported acceptance summaries use infinity norms and row-specific thresholds. Those quantities should not be interchanged.

A trust region limits a local model's proposed step, while bounds restrict candidates to admissible ranges. These mechanisms can improve control of a local search, but they do not make every trial economically meaningful or guarantee that every starting point will succeed. EconForge checks domains and finite values at required evaluations. A failed evaluation remains visible; the adapter introduces neither a hidden restart nor a replacement residual intended to conceal the failure.

4.3 Termination and verification are different decisions

Figure 2 shows how a single explicit attempt is assessed. Backend output must have expected types, dimensions, finite values, and consistent callback counts. A successful termination flag is necessary for a passing attempt, but it is not sufficient. A fresh scalar evaluation checks the returned candidate against every original equation, including reporting identities, and compares its canonical residual with the backend's cached value. The scalar evaluator is separate from the AD path, although both use the same declared model. This is computational separation, not an independent formulation of economic theory.

The default operational limits allow at most 16 explicitly supplied starts, 2,000 residual-budget charges per start, and 60 seconds per start. AD requests incur conservative budget charges for scalar admission and differentiated evaluation. These charges are not optimizer iterations. Deadlines are checked around callbacks and kernels; they are not guaranteed interruption of an in-progress native call. The public result can report an unavailable iteration count, and this paper does not reconstruct an iteration history from callback events.

When multiple starts are requested, all attempts are retained. Accepted points are grouped using a fixed comparison rule in scaled coordinates. Distinct observed branches require an explicit selection, and a failed requested attempt is not erased by a successful companion. The result therefore communicates both numerical information and the limits of the request. In particular, a successful root leaves model_status as NOT_EVALUATED: satisfying equations is deliberately separated from endorsing the model.

5 Computational experimental design

The research protocol was recorded before manuscript drafting. The primary artifact is the published wheel, independently downloaded alongside the source distribution. Their recomputed SHA256 digests match the release identities in Appendix B. Experiments install that wheel in a new environment and import the installed package. Public source extracted from the release provides the inspected implementation. No accepted source, frozen specification, original Git reference, or private scientific record was modified.

The observed environment is macOS 26.5 on arm64, CPython 3.12.14, NumPy 2.2.6, SciPy 1.15.3, JAX and jaxlib 0.6.2, ml_dtypes 0.6.0, and opt_einsum 3.4.0. These fields match the documented qualified stack. NumPy supplies array operations (Harris et al., 2020). CPU execution, 64-bit arithmetic, disabled JIT, and single-thread environment settings were explicitly recorded. This is a fresh local reproduction in that configuration, not new cross-platform certification. The unmodified packaged RBC quickstart, NK quickstart, and five-case example runner each exited successfully; their repeated cases do not enlarge the count of distinct benchmarks.

Table 2. Released calibrations and initial configurations.

Model	Parameters	Canonical unknowns	Documented starts
RBC-A	$\alpha = 1/3, \beta = .96$ $\rho = .9, \delta = 1$ $\gamma = 1$	(K, z, c)	Primary $(.8K^*, 0, 1.2c^*)$ robustness $(1.2K^*, 0, .8c^*)$
RBC-B	$\alpha = .36, \beta = .99$ $\rho = .9, \delta = .025$ $\gamma = 2$	(K, z, c)	Primary $(.8K^*, 0, 1.2c^*)$ robustness $(1.2K^*, 0, .8c^*)$
NK-A	$\beta = .99, \sigma = 1$ $\kappa = .1, \phi_\pi = 1.5$ $\phi_x = .5, \rho_v = .5$	(v, x, π)	$(.001, .001, .001)$

The five released configurations constitute the replication set. The two RBC starts share equations and calibration within each model, so five runs do not represent five distinct economic theories. Analytical references are recomputed from (7), (11), and (14), rather than copied from numerical outputs. The original release also provides reference values; agreement with them is recorded separately. Where algebraically equivalent expressions evaluate in a different floating-point order, tiny differences are expected and retained.

The primary measurements are initial and final residual norms, all original-row acceptance results, coordinate errors, AD Jacobians, and complete structured diagnostic outputs. The analytical comparison follows the released scale convention:

$$e_j = |\hat{w}_j - w_j^*| / d_j, \quad e_j \le 10^{-8} + 10^{-6} |w_j^* / d_j|. \quad (17)$$

For the reported error in (17), the reference is the independently computed closed form rounded to binary64; it is not an unrounded exact-real oracle. This is separate from the residual threshold in (2). For NK's zero reference, the absolute component remains meaningful. For RBC, declared capital and consumption scales make the errors comparable within the benchmark convention. Raw coordinate errors remain in the data files, so the scaling choice is auditable.

Additional analyses are bounded and explicitly exploratory. Residual slices hold productivity at zero and evaluate formulas over a positive neighborhood of the reference. An analytical derivative implementation checks JAX values at selected points. Controlled invalid or limited requests test whether an unsuitable result is suppressed. The purpose is to explain behavior and detect specific inconsistencies, not to estimate failure probabilities over a population of models. The experiment manifest records inputs and criteria, and the raw logs preserve unsuccessful outcomes.

6 Results

6.1 Reproduction of the released configurations

All five released configurations returned PASS, with successful backend termination and fresh original-equation verification. All componentwise analytical-reference tests also passed. The largest raw canonical residual was 2.1956×10^{-12}, and the largest row-normalized residual was 1.8658×10^{-12}, both in RBC-B primary. The largest scaled reference error was 9.5386×10^{-11} in that same case. Table 3 reports each outcome rather than

only a pooled maximum. All three canonical Jacobians are 3×3; NK additionally retains its Taylor reporting row in scalar verification.

Table 3. Recorded results for the five released configurations.

Case	Final raw norm	Final normalized norm	Maximum scaled error	Result
RBC-A primary	4.441e-16	1.708e-16	1.533e-16	PASS
RBC-A robustness	5.954e-20	5.954e-20	5.954e-19	PASS
RBC-B primary	2.196e-12	1.866e-12	9.539e-11	PASS
RBC-B robustness	2.776e-17	2.106e-16	5.985e-15	PASS
NK-A	7.941e-23	7.941e-23	2.118e-22	PASS

All norms are canonical infinity norms. Error uses equation (17). Exact coordinate arrays and original-row records are retained in the accompanying JSON and CSV.

6.2 Geometry and derivatives

Figure 4 evaluates RBC-A near its analytical root, holding $z = 0$. The resource restriction relates feasible consumption to capital, whereas the Euler restriction fixes the stationary marginal return. Their crossing explains why both equations matter. A feasible resource allocation alone need not satisfy intertemporal equilibrium. The displayed neighborhood contains positive capital and consumption; extending it across a domain boundary would invalidate the economic and numerical interpretation.

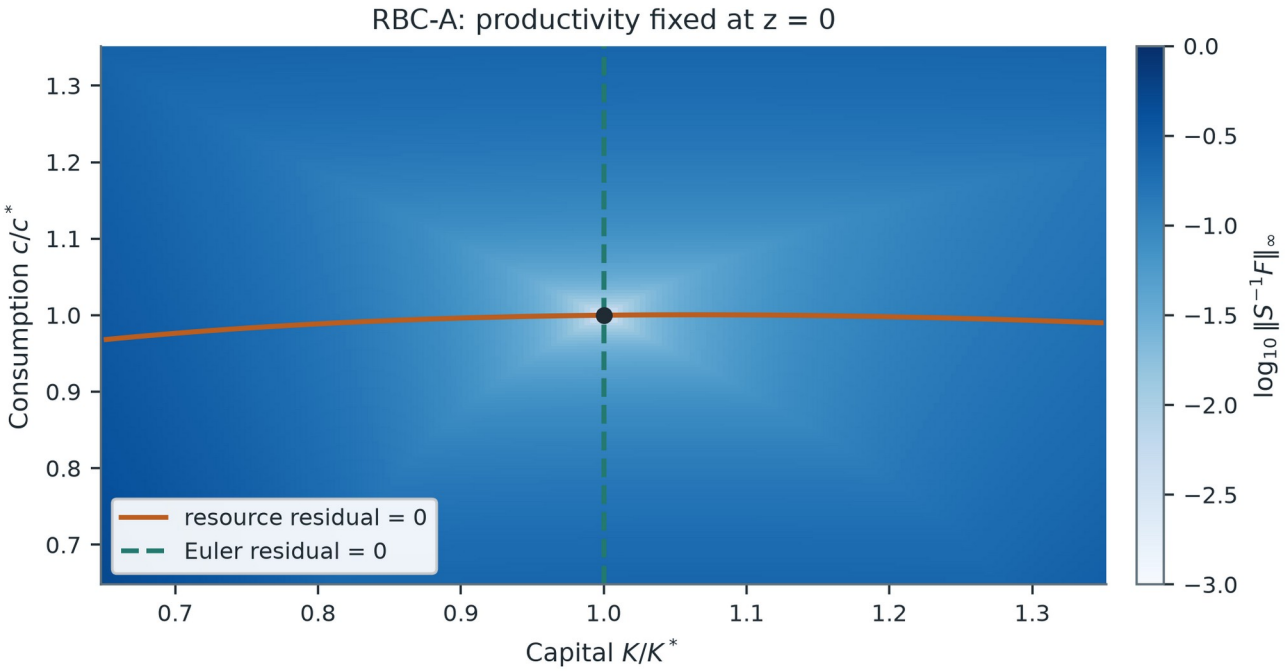


Figure 4. RBC-A local residual geometry. Local RBC-A residual geometry at z = 0. The orange resource-balance locus intersects the dashed Euler locus at the analytical reference (black point). Color shows the normalized infinity norm on a positive K–c grid. The color scale saturates below 10^{-3} to keep the surrounding geometry legible; the CSV retains every raw value. This slice does not establish global solver behavior.

Figure 5 repeats the local analysis for RBC-B. Partial depreciation changes the resource relation to $c = K^{\alpha} - \delta K$, while the Euler restriction includes the surviving capital return. The larger capital reference and different

residual scale are consequences of the stated calibration and normalization. The figure does not isolate a causal effect of CRRA curvature, because other parameters differ from RBC-A. The two panels are distinct one-dimensional slices: the resource-balance locus and the Euler residual at fixed reference consumption. Neither establishes that one model is harder to solve.

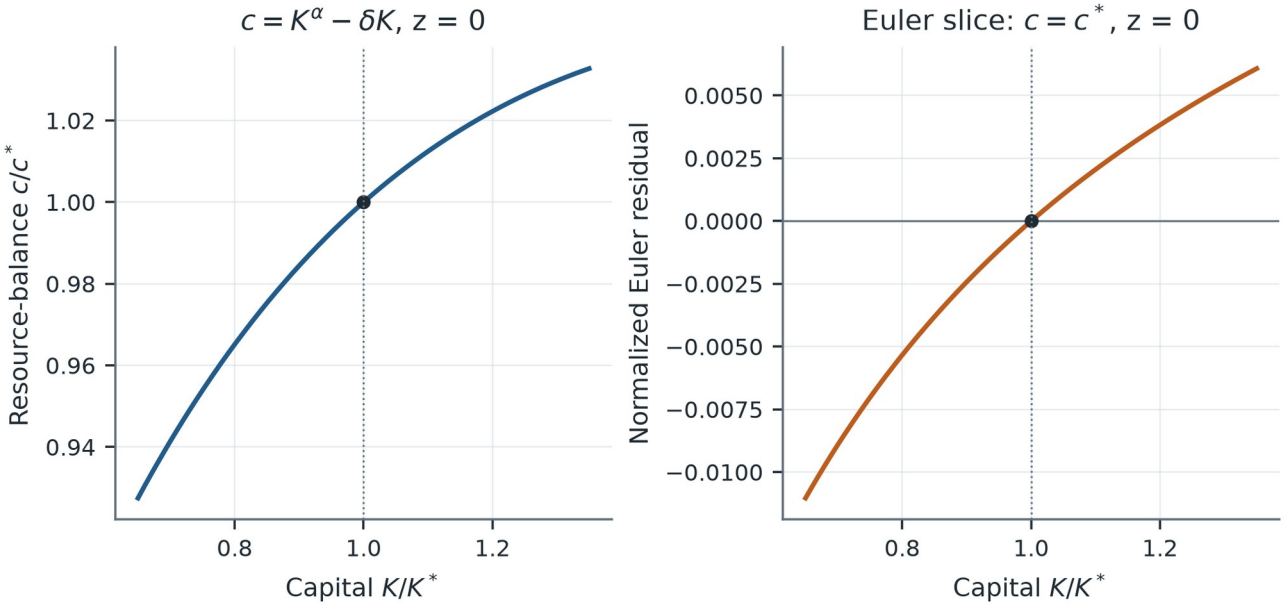


Figure 5. RBC-B resource balance and Euler residual. RBC-B equilibrium structure with partial depreciation and CRRA marginal utility. Left: consumption compatible with resource balance. Right: the normalized Euler residual when consumption is fixed at its analytical reference. Both mark K = K*. The two panels are distinct slices; neither is a solver trajectory or an empirical production estimate.

Figure 6 uses elimination to display the NK equilibrium. At $\nu = 0$, the stationary Phillips and policy-substituted IS relations intersect at the origin. The included policy relationship therefore enters through the eliminated nominal-rate alias. This construction avoids portraying three unknowns as if they were an unrestricted two-variable system. The nonzero initial guess demonstrates a numerical return to the homogeneous reference, not a monetary-policy experiment.

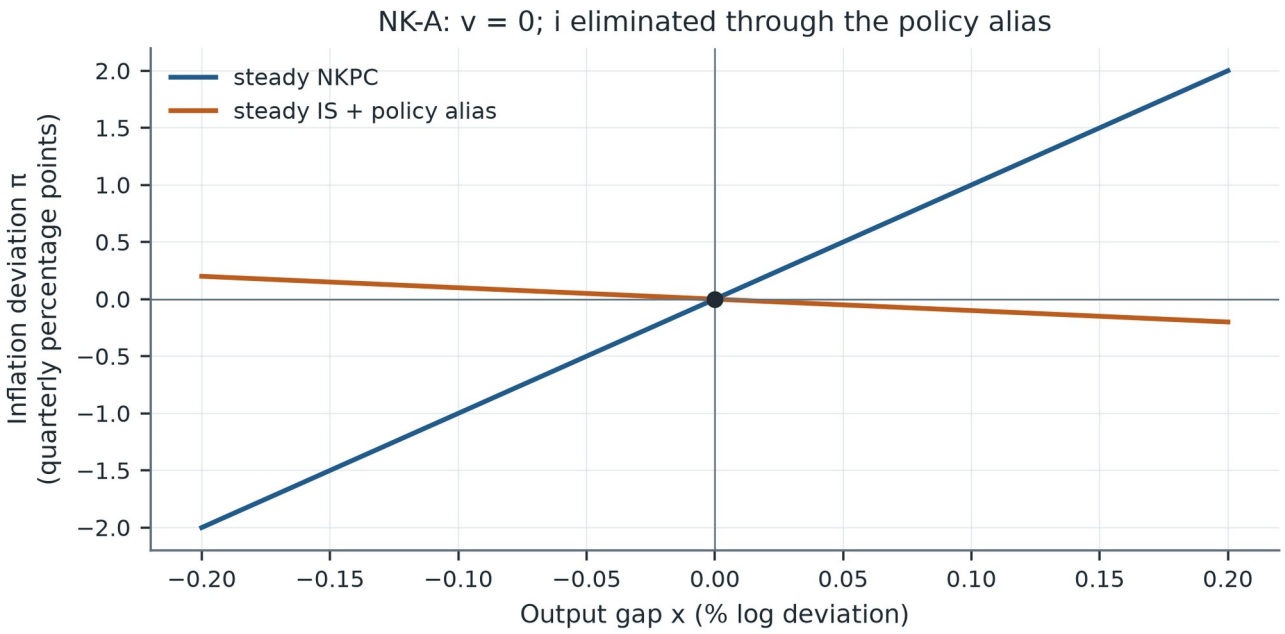


Figure 6. NK-A steady equilibrium relationships. NK-A after steady substitution, v = 0 and elimination of the policy-rate alias i. The NKPC gives $\pi = \kappa x/(1-\beta)$, whereas the IS relationship with the policy alias gives $\pi = -\varphi_x x/(\varphi\pi - 1)$. Their intersection is the zero steady state. Multiplication by 100 only changes the displayed units; inflation remains a quarterly rate deviation.

Figure 7 reports the measured dimensionless matrices $D_S^{-1} J D_d$ for all three models, with labeled economic equations and stored variables. The following examples explain the underlying raw derivatives; the displayed entries additionally incorporate the declared row and variable scales. A Jacobian entry measures the local change in one residual when one stored variable changes, holding others fixed. For example, the RBC resource residual has derivative one with respect to consumption: additional consumption creates a one-for-one resource imbalance. At an interior RBC root, the Euler row's consumption derivative is zero because the bracket multiplying marginal utility vanishes. This does not imply that consumption is economically irrelevant; the resource equation still determines its steady value.

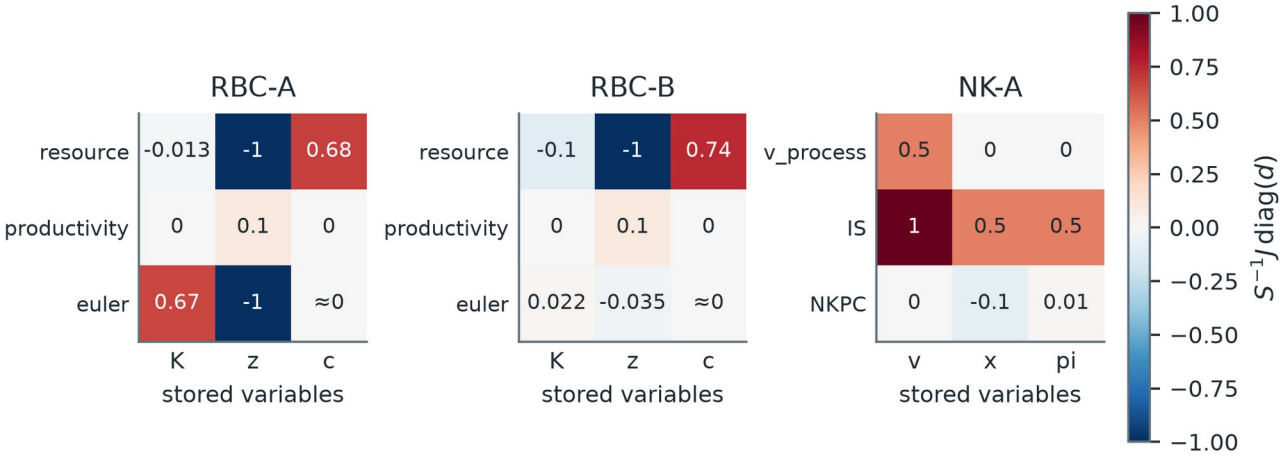


Figure 7. Dimensionless Jacobians at computed steady candidates. Dimensionless steady Jacobians $S^{-1}J\,\mathrm{diag}(d)$, formed from the measured automatic-differentiation matrices at the computed primary solutions. Rows are canonical residual equations and columns are stored variables. Scaling makes sensitivities comparable within the declared model scales. Nonzero entries smaller than 10^{-10} are labeled ≈0; the CSV retains their measured values. The NK reporting identity is absent from the 3 × 3 canonical matrix but remains in scalar verification.

The analytical and JAX Jacobians agreed at all five computed candidates under the stated dimensionless comparison criterion. The largest entrywise discrepancy in $D_S^{-1}(J_{AD} - J_{analytic})D_d$ was 7.1166×10^{-16}. This checks five points, not differentiability or derivative accuracy across an entire domain. Separately evaluated algebraic residual formulas also agreed to the retained floating-point discrepancies; small differences from the literal expression evaluation are not silently replaced by exact zeros.

6.3 Accuracy relative to analytical references

Figure 8 compares initial and final row-normalized residual norms. The common normalized presentation avoids directly comparing a marginal-utility residual with a goods-balance residual in arbitrary units. All five measured norms are positive, so the logarithmic axis uses no artificial floor. The figure is an endpoint comparison. It contains no claim about iteration-by-iteration convergence or computational speed.

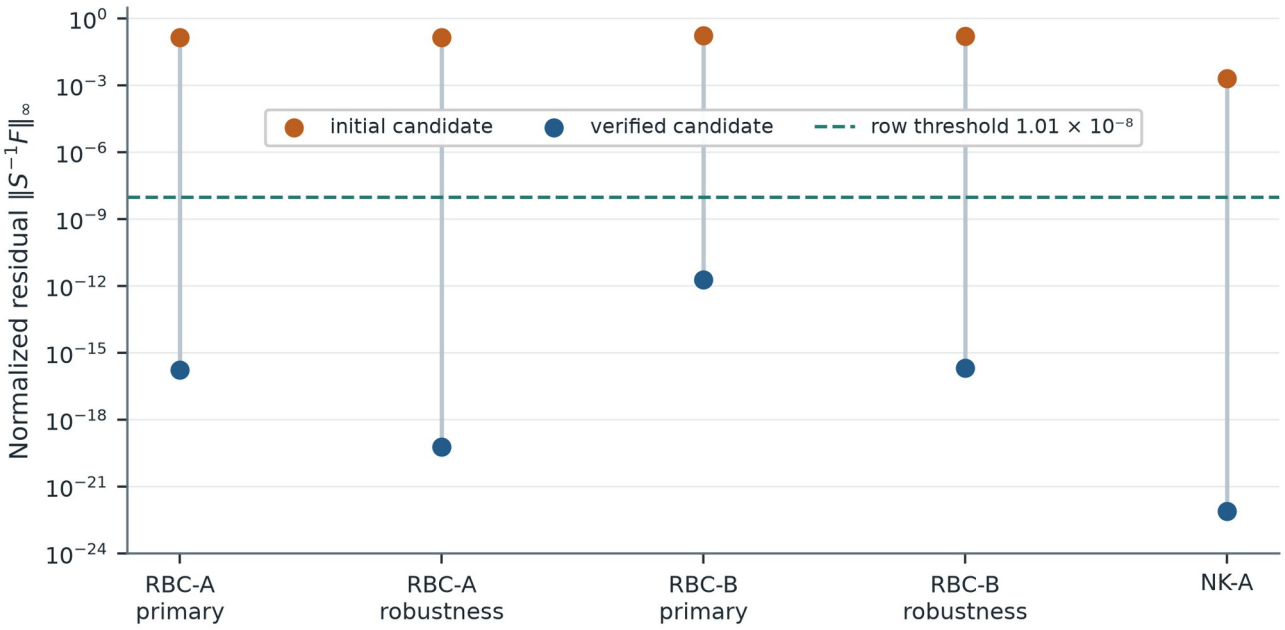


Figure 8. Initial and verified residual norms. Initial and final normalized infinity norms for the five released cases. All measured final norms lie below 1.01×10^{-8}, the common normalized row threshold in these fixtures. Gray segments connect endpoint values only. No exact-zero norm occurs in this run, so the logarithmic chart requires no artificial floor; each individual residual is preserved in the data.

Figure 9 compares numerical candidates with binary64-rounded independent analytical references using the declared variable scales. The closed forms are evaluated with 60-digit decimal arithmetic from the decimal representations of the declared parameters, then rounded for this comparison. Tiny coordinate differences are not claims of accuracy beyond floating-point resolution. This check answers a different question from residual acceptance: whether the returned point agrees with the known benchmark solution. The analytical comparison is especially useful because a program could consistently evaluate and solve an incorrectly transcribed equation. Here, the derivation and separately written scalar formulas provide a second route to the expected values. Their independence is limited to the implementation path and mathematical calculation; this is still an internal research audit.

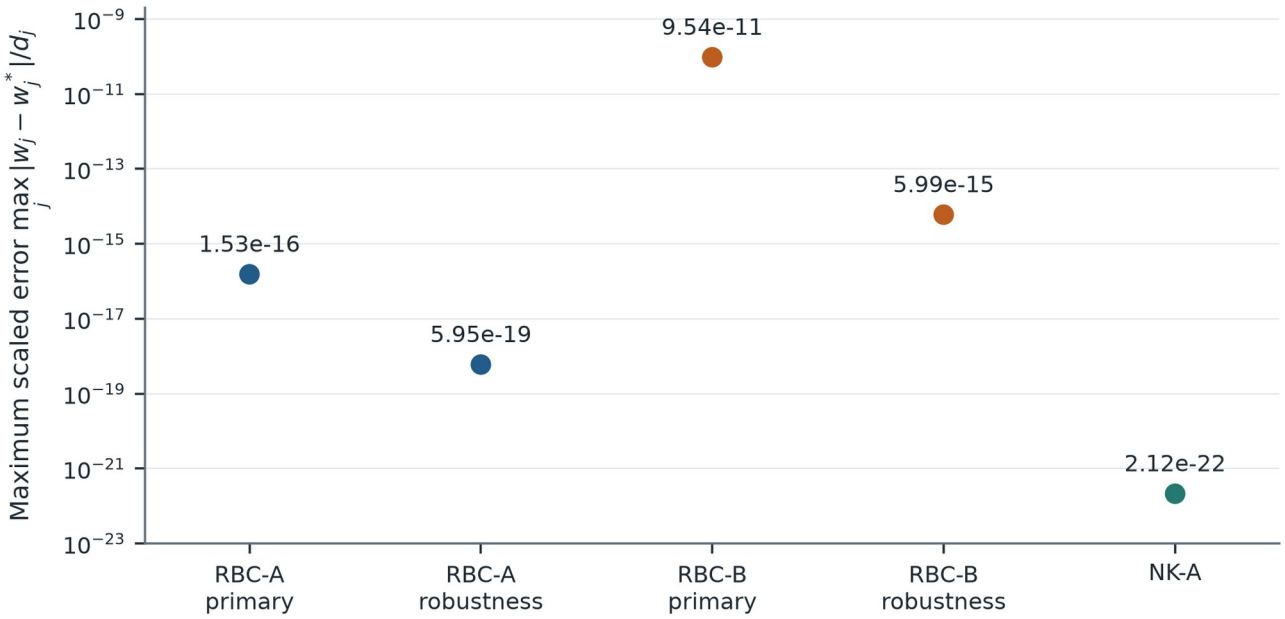


Figure 9. Numerical error against independent closed forms. Maximum component error relative to binary64-rounded, separately derived closed-form references, divided by each declared variable scale. The largest value is approximately 9.54×10^{-11} for RBC-B primary. NK scales equal one, avoiding division by a zero reference. These errors are direct data comparisons, not estimates read from a graph; all released componentwise reference tests pass.

6.4 Starting points and conservative outcomes

Figure 10 displays the released starts and their recorded outcomes. The paired RBC starts use opposing capital-consumption perturbations and return candidates agreeing with the same analytical reference. This is useful evidence about these particular starts. It is not a map of the entire attraction region, and the word robustness in a fixture name does not broaden the claim. NK has only one released nonzero start in the replication set.

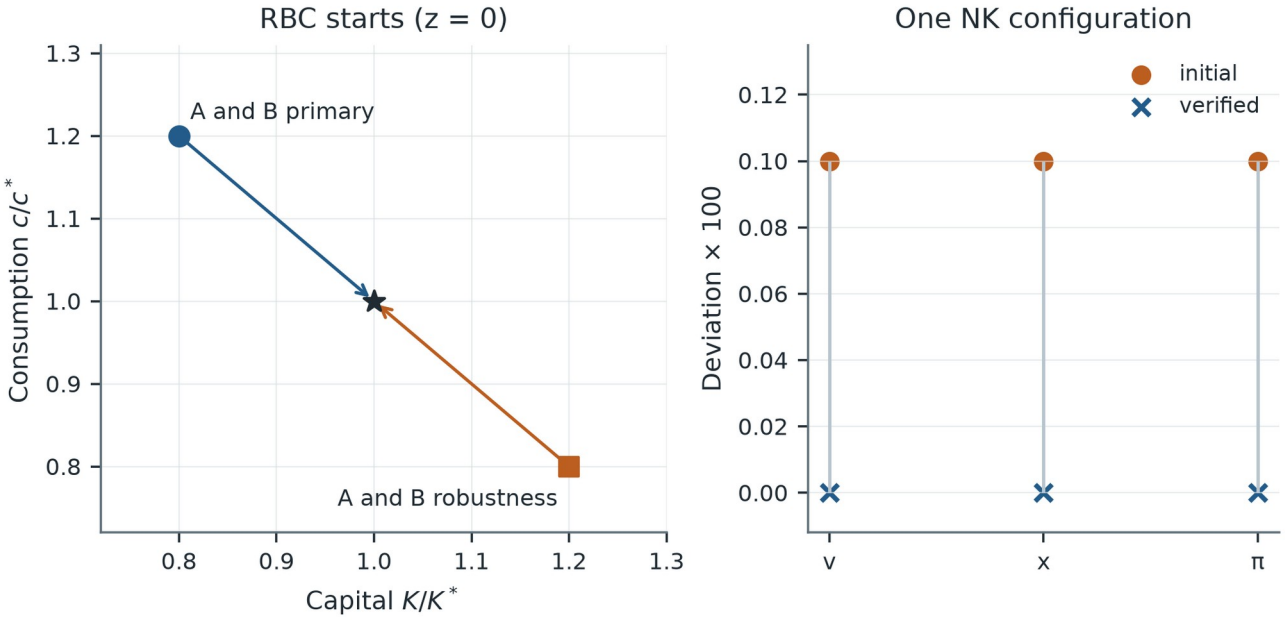


Figure 10. Released starting points and final candidates. Released starting configurations and measured endpoints. RBC-A and RBC-B share the same two normalized starting pairs, (0.8, 1.2) and (1.2, 0.8), with z initially zero; all four final pairs coincide visually at (1, 1). The NK start has $v = x = \pi = 0.001$ and ends near zero. Arrows and segments compare endpoints only, without asserting an iteration path or a global attraction region.

Eleven additional solve requests were executed. Six broader explicit starting vectors passed, and a request containing the two documented RBC-A starts passed with one observed cluster. Four controlled requests failed: a one-evaluation backend budget, zero capital, a missing start, and a request combining one valid and one invalid start. Their actual aggregate status is FAIL, not REFUSED. The mixed request retains a converged and verified individual attempt while suppressing an aggregate public solution. These are input and budget controls, not mutations of the implementation, and they do not establish failure rates for other models.

Figure 11 uses actual recorded outcomes to separate the logical roles of termination, fresh residual checks, and final numerical acceptance. A candidate that satisfies a row tolerance still needs the rest of the requested procedure to complete. Successful optimizer termination is not, in general, a proof that the original equations pass. The controls here demonstrate budget and input failures and a mixed aggregate request; they do not exhibit a converged backend followed by failed fresh residual verification. No controlled input is presented as an accepted release benchmark. Economic validation remains outside the acceptance pipeline.

Observed request	≥1 converged attempt	≥1 verified attempt	Solution exposed
Five released cases	YES	YES	YES
RBC-A evaluation cap = 1	NO	NO	NO
RBC-A K = 0	NOT RUN	NOT RUN	NO
RBC-A no explicit start	NOT RUN	NOT RUN	NO
RBC-A valid + invalid start	YES	YES	NO

Figure 11. Observed verification and request outcomes. Observed separation of convergence, fresh scalar verification and public-solution exposure. The first row summarizes each of the five released cases. The remaining rows are new negative controls. A valid and an invalid start in one request produce at least one converged, verified attempt but an overall failed request with no public solution. NOT RUN means the corresponding stage was never reached; it is not a measured residual failure.

7 Discussion

The evidence supports a limited but useful conclusion: the released workflow can connect specified equations, explicit starts, automatic derivatives, a numerical solver, and fresh residual checks in a reproducible record. Its educational value lies in making those connections inspectable. Students can compare an analytical capital formula with a returned candidate, inspect how the Jacobian depends on coordinates, and observe why a nominal success flag is only one part of a numerical assessment.

The design also has costs. Explicit domains and conservative refusals can reject a request that a more permissive solver might repair or continue. Frozen tolerances help make records comparable, but their appropriateness still depends on the declared scales and the research question. Recording rich metadata increases complexity. These tradeoffs should be evaluated against the goal of transparent scientific evidence, rather than interpreted as proof that stricter behavior always produces a better economic answer.

Reproducibility requires more than rerunning a command. Figure 12 connects distribution identity, environment, model input, output, and verification. A different wheel with the same displayed version is a different artifact. A matching wheel on a new platform is a new observation until its behavior has been evaluated. A copied

analytical target without a derivation is weaker evidence than a separately justified formula. The package therefore retains the original model files, downloaded distributions, explicit commands, data, and provenance alongside the paper (Sandve et al., 2013; Smith et al., 2016).

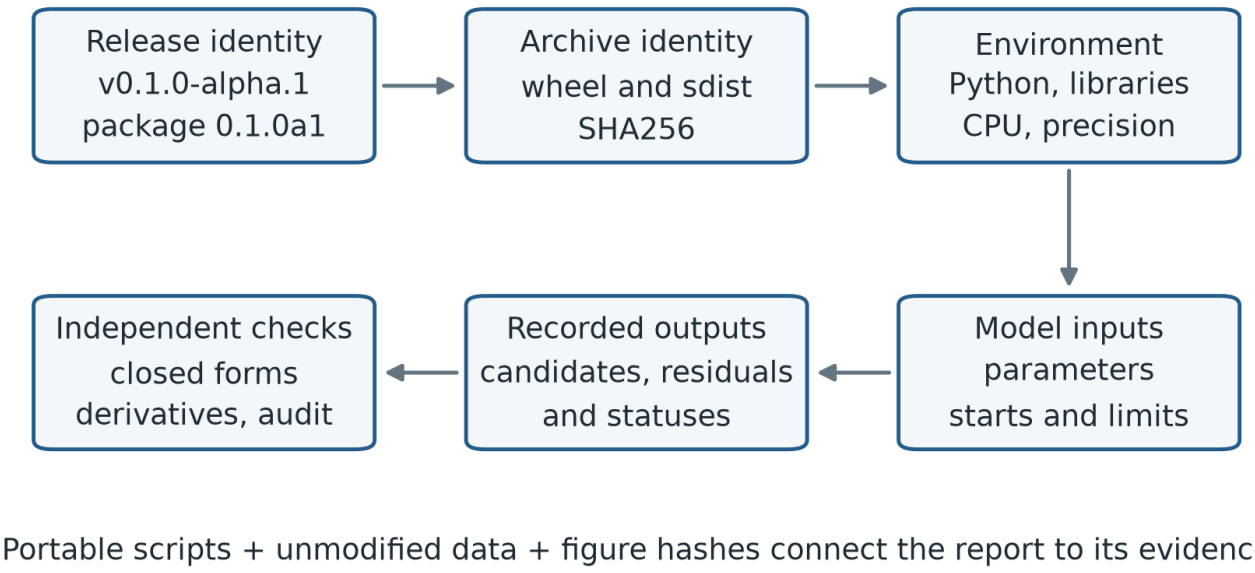


Figure 12. Reproducibility and evidence chain. Evidence chain for reproduction. Version identifiers and archive hashes identify the software, while recorded environments and explicit inputs identify an execution. Saved outputs are checked against separately derived formulas and linked to figures through data and output hashes. This documentation supports inspection and rerunning, not a claim of universal reproducibility.

The strongest mathematical statements here concern the stated equations. For example, the calibrated NK linear system has a unique stationary root because its coefficient matrix is nonsingular. The RBC positive stationary capital formulas follow from the Euler and resource restrictions under their assumptions. These derivations do not transfer automatically to a nonlinear custom model merely because it uses the same parser. In contrast, the software evidence establishes concrete behavior: which inputs were accepted, which records were returned, and which tolerances were met.

The study makes no performance-superiority claim. Its library methods are established, the experiments are small, and no controlled external benchmark was run. Relative to broader systems such as Dynare, EconForge is best understood here as a narrowly scoped, inspectable workflow (Dynare Team, n.d.). The potential contribution is the clarity of the integration and evidence, not a priority claim or a new mathematical result. Human reviewers should judge whether that contribution is useful and sufficiently explained for the intended educational publication.

8 Limitations and future work

Five released configurations cover three small deterministic steady-state systems. They do not sample a broad class of nonlinear equations, difficult parameter regions, or large macroeconomic models. Several successful starts cannot establish universal solver correctness or global convergence. Residual tolerances do not certify forward error for ill-conditioned systems, and a numerical rank diagnostic is not a uniqueness theorem. The separate analytical examples help precisely because their mathematical structure is unusually tractable.

The platform evidence is also narrow. The present runs record a specific local macOS Apple Silicon environment and preserve its actual versions. They do not extend the release's qualification to Linux or Windows, and no successful Linux H recovery is claimed. Historical private acceptance is not treated as

independent public certification. Advanced dynamic and stochastic modules, estimation, stochastic simulation, and higher-order or nonlinear impulse responses are outside this paper's supported scope. P3.10 is not claimed as implemented.

Economic limitations are equally important. Fixed labor, selected calibrations, and deterministic substitution are teaching simplifications. Neither RBC example estimates preferences or technology from observations. NK-A's zero gap reference does not test a central bank's actual policy or establish dynamic determinacy. A root of the implemented equations is not automatically a welfare optimum, an empirically adequate model, or a valid stochastic equilibrium.

Future work could expand independently derived examples, examine difficult but well-defined parameter regimes, and repeat the same evidence protocol on additional platforms. Broader software comparisons would require matched equations, versions, settings, and acceptance criteria. Any extension should retain negative outcomes and report its new scope explicitly. Before publication, a human reviewer must also assess the source interpretations, mathematical explanations, authorship, and AI-assistance disclosure. The internal adversarial audits supplied here are preparation for that review, not a substitute for it.

9 Conclusion

EconForge 0.1.0a1 illustrates a transparent route from model text to qualified steady-state results. Reconstructing its actual RBC and NK equations, independently deriving references, and retaining numerical diagnostics makes the small benchmark study checkable. The five reproduced configurations support the reported local computations under their recorded environment. They do not certify arbitrary models or the broader economic claims that a root cannot answer. The manuscript and accompanying evidence package offer a bounded software-methods contribution: a reproducible explanation of what was computed, why the expected answer is justified, and how the result was checked.

Authorship and assistance disclosure

The author is Finley Dou, affiliated with Shanghai High School International Division. The project owner commissioned this manuscript. AI assistance was used for source inspection, literature discovery and verification, mathematical exposition, experimental scripts, figures, drafting, formatting, and internal adversarial audits. The human author must review the scientific content, source interpretations, numerical evidence, and final submission, and adapt this disclosure to the eventual website's editorial policy. No grant support, external peer review, acceptance, or publication is asserted.

References

[1] Kydland, Finn E.; Prescott, Edward C. (1982). Time to Build and Aggregate Fluctuations. Econometrica, 50(6), 1345–1370. DOI 10.2307/1913386. Accessible reading copy.

[2] King, Robert G.; Plosser, Charles I.; Rebelo, Sergio T. (1988). Production, Growth and Business Cycles: I. The Basic Neoclassical Model. Journal of Monetary Economics, 21(2–3), 195–232. DOI 10.1016/0304-3932(88)90030-X. Accessible reading copy.

[3] Plosser, Charles I (1989). Understanding Real Business Cycles. Journal of Economic Perspectives, 3(3), 51–77. DOI 10.1257/jep.3.3.51.

[4] Clarida, Richard; Galí, Jordi; Gertler, Mark (1999). The Science of Monetary Policy: A New Keynesian Perspective. Journal of Economic Literature, 37(4), 1661–1707. DOI 10.1257/jel.37.4.1661. Accessible reading copy.

[5] Taylor, John B. (1993). Discretion versus Policy Rules in Practice. Carnegie-Rochester Conference Series on Public Policy, 39, 195–214. DOI 10.1016/0167-2231(93)90009-L. Accessible reading copy.

[6] Judd, Kenneth L. (1998). Numerical Methods in Economics. MIT Press. Official source.

[7] Branch, Mary Ann; Coleman, Thomas F.; Li, Yuying (1999). A Subspace, Interior, and Conjugate Gradient Method for Large-Scale Bound-Constrained Minimization Problems. SIAM Journal on Scientific Computing, 21(1), 1–23. DOI 10.1137/S1064827595289108.

[8] SciPy Developers (2025). scipy.optimize.least_squares — SciPy 1.15.3 Manual. SciPy. Official source.

[9] Virtanen, Pauli; Gommers, Ralf; Oliphant, Travis E., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17, 261–272. DOI 10.1038/s41592-019-0686-2.

[10] The JAX Authors (n.d.-a). Higher-order Derivatives. JAX documentation. Official source.

[11] The JAX Authors (n.d.-b). Default Dtypes and the X64 Flag. JAX documentation. Official source.

[12] Baydin, Atılım Güneş; Pearlmutter, Barak A.; Radul, Alexey Andreyevich, et al. (2018). Automatic Differentiation in Machine Learning: A Survey. Journal of Machine Learning Research, 18(153), 1–43. Official source. Accessible reading copy.

[13] Sandve, Geir Kjetil; Nekrutenko, Anton; Taylor, James, et al. (2013). Ten Simple Rules for Reproducible Computational Research. PLOS Computational Biology, 9(10), e1003285. DOI 10.1371/journal.pcbi.1003285.

[14] Wilson, Greg; Aruliah, D. A.; Brown, C. Titus, et al. (2014). Best Practices for Scientific Computing. PLOS Biology, 12(1), e1001745. DOI 10.1371/journal.pbio.1001745.

[15] Smith, Arfon M.; Katz, Daniel S.; Niemeyer, Kyle E., et al. (2016). Software Citation Principles. PeerJ Computer Science, 2, e86. DOI 10.7717/peerj-cs.86. Accessible reading copy.

[16] Peng, Roger D. (2011). Reproducible Research in Computational Science. Science, 334(6060), 1226–1227. DOI 10.1126/science.1213847. Accessible reading copy.

[17] Harris, Charles R.; Millman, K. Jarrod; van der Walt, Stéfan J., et al. (2020). Array Programming with NumPy. Nature, 585, 357–362. DOI 10.1038/s41586-020-2649-2. Accessible reading copy.

[18] Dynare Team (n.d.). The Dynare Reference Manual, Version 7.2. Dynare. Official source. Accessible reading copy.

[19] Dou, Finley (2026). EconForge 0.1.0a1 — Experimental Alpha [Computer software]. GitHub Releases. Official source. Accessed 28 October 2026.

Appendix A Derivatives and numerical settings

For the generalized RBC steady system in (10), define $q=\alpha e^{z}K^{\alpha-1}$, $Y=e^{z}K^{\alpha}$, and $R=q+1-\delta$. The separately derived raw Jacobian is

$$J_B=\begin{pmatrix} \delta-q & -Y & 1 \\ 0 & 1-\rho & 0 \\ \beta(1-\alpha)qc^{-\gamma}/K & -\beta q c^{-\gamma} & -\gamma c^{-\gamma-1}(1-\beta R) \end{pmatrix}. \quad (18)$$

RBC-A is the specialization $\delta=\gamma=1$ of these steady formulas, consistent with its literal source expressions. The expression is a derivative in stored levels, not in log coordinates. At a root, $1-\beta R=0$ explains the vanishing lower-right entry. Equation (15) gives the NK derivative independently of JAX.

Table 4. Numerical settings and distinct acceptance obligations.

Item	Recorded setting or criterion
Algorithm	SciPy least_squares 1.15.3; TRF; linear loss; exact subproblem option
Derivative	Callable JAX 0.6.2 AD; CPU; float64; no JIT
Optimizer tolerances	ftol = xtol = gtol = 10^{-12}
Residual contract	$\lvert F_i \rvert \le S_i 10^{-10} + L_i 10^{-8}$; all original rows
Reference comparison	$\lvert \widehat{w}_j - w_j^* \rvert / d_j \le 10^{-8} + 10^{-6} \lvert w_j^* / d_j \rvert$
Analytical AD comparison	$\lvert \Delta_{ij} \rvert \le 10^{-7} + 10^{-5} \lvert \left(D_S^{-1} J_{analytic} D_d \right)_{ij} \rvert$
Default caps	16 explicit starts; 2,000 residual charges; 60 seconds per start
Interpretation	Canonical F excludes identity rows; original-row checks retain them

Here $\Delta = D_S^{-1} \left(J_{AD} - J_{analytic} \right) D_d$ is the dimensionless matrix of derivative discrepancies.

The released AD validation policy specifies dimensionless derivative tolerances 10^{-7} and 10^{-5}. The additional analytical-Jacobian comparison in this paper uses those same constants. This independent comparison is distinct from production AD admission, which checks finite values and scalar/JAX primal agreement, and from the optimizer and residual tolerances. The experiment records specify which criterion applies to each measurement. The release's defaults and the reduced caps used for negative tests remain unchanged in the installed software.

Appendix B Artifact identities and reproduction

The release page is EconForge v0.1.0-alpha.1.

Package version: **0.1.0a1**. Public tag commit: `a792375b3828c4e29776d3552098dd48f36bcc75`. The public release commit is distinct from the internal accepted scientific candidate. The wheel and source archive contain 55 byte-identical top-level Python modules. Their identity checks establish artifact correspondence, not mathematical certification.

econforge-0.1.0a1-py3-none-any.whl

SHA256: `d9301e86c50da97b1e4a475ed937693b340416f6f9d35d444afbcb31e1df1b26`

econforge-0.1.0a1.tar.gz

SHA256: `1d1684d12fb65f72b4b69eef4ecaa6662ca7559c659249bbf96411680c33407f`

Copy the research folder to a fresh location before rerunning, preserving the original evidence. From that copy, use a CPython 3.12.14 interpreter:

```
ECONFORGE_PYTHON=/path/to/python3.12 sh experiments/scripts/reproduce.sh
```

The script installs the exact bundled release wheel with its pinned AD dependencies, checks artifacts, runs the packaged examples and research experiments, records exit codes, and audits the outputs. Network access is needed when dependencies are not cached. For figures, create a separate plotting environment from experiments/scripts/plot-requirements.txt, then run experiments/scripts/plot_figures.py, which invokes plot_empirical.py internally. The plotting environment is separate from the solver environment; its versions appear in evidence/PLOTTING_ENVIRONMENT.json.

The runner reads installed examples and preserves exact parsed input bytes separately from the original packaged JSON. Whitespace reserialization changes a text hash without changing model numbers or expressions, so both identities are recorded. Human-readable reproduction instructions are in experiments/README.txt. The public figures and manuscript contain no private development evidence.

The evidence directory provides the claim-evidence matrix, experiment manifest, software identities, and figure provenance. The references directory contains the source-verification register and BibTeX bibliography. The reports directory records mathematical, scientific, and document-render audits. Website Markdown uses relative figure paths and remains an unpublished adaptation for later human editorial review.

Appendix C Public implementation locators

All paths refer to the inspected public release. Model parsing defines validation and canonical ordering; steady transformation defines dated-variable substitution. The model equations and exact values are in examples/RBC-A_primary.json, examples/RBC-B_primary.json, examples/NK-A.json and examples/cases.json.

Residual evaluation checks all original rows; automatic Jacobians defines the AD transformation and point checks. Solver dispatch fixes the least-squares options.

Operational limits, fresh verification, and attempt aggregation support the precise behavior described in Section 4. These source locations establish implementation behavior, not an external mathematical proof.